Práctica: Desarrollo de un Sistema de Información mediante Spec-Driven Development

Enunciado, metodología de trabajo y entregables

1. Presentación y objetivos

Esta práctica tiene como objetivo que los alumnos experimenten de primera mano el desarrollo de una aplicación web real siguiendo una metodología de Spec-Driven Development (SDD), en la que la especificación funcional —y no el código— es el artefacto que dirige todo el proceso de construcción del software.

Cada grupo recibirá la descripción detallada de un sistema de información distinto, redactada en lenguaje natural. A partir de esa descripción, el grupo deberá identificar las historias de usuario implícitas, formalizarlas, planificar su implementación técnica, descomponerlas en tareas y, finalmente, construir una aplicación funcional que las resuelva, iterando tantas veces como sea necesario hasta obtener un resultado satisfactorio.

Objetivos de aprendizaje

- Practicar la elicitación de requisitos a partir de una descripción funcional en lenguaje natural, sin historias de usuario explícitas.
- Aplicar de forma completa el ciclo de Spec-Driven Development: especificar, planificar, descomponer en tareas e implementar.
- Experimentar el trabajo iterativo (loop engineering), refinando la especificación, el plan o la implementación cada vez que el resultado obtenido no sea satisfactorio.
- Coordinar un equipo de desarrollo de cuatro personas con responsabilidades individuales claras y trazables.
- Producir una aplicación web funcional que respete, en la medida de lo posible, la arquitectura propuesta para el sistema asignado.
- Refinar una aplicación ya implementada mediante clarify, analyze y converge, justificando las decisiones y verificando su efecto en el comportamiento del sistema.

2. Grupos de trabajo y material de partida

La práctica se realiza en grupos de 2 personas. Cada grupo será asignado a uno de los sistemas de información descritos en el documento «Descripción detallada del sistema», que constituye el único punto de partida funcional: no se entregará a los grupos el listado de historias de usuario ni el documento de detallado de arquitectura propuesta correspondientes a ese sistema, ya que forman parte del trabajo que el propio grupo debe realizar.

El primer reto del grupo consiste, por tanto, en leer con atención esa descripción y extraer de ella las historias de usuario que están implícitas en el texto: quién realiza cada acción, qué necesita conseguir y qué condiciones debe cumplir el sistema para darla por buena. Se espera que cada grupo identifique un conjunto de historias equivalente en número en torno a 8 historias de usuario.

3. Metodología: Spec-Driven Development

El desarrollo de la aplicación debe seguir el ciclo de Spec-Driven Development en sus cuatro fases características. Estas fases no son un trámite burocrático previo a «programar de verdad»: son el propio proceso de desarrollo, y de su calidad depende directamente la calidad del software que se obtiene al final.

Specify (especificar)

Por cada historia de usuario identificada, el grupo redacta una especificación clara: qué se quiere conseguir, quién lo pide, qué reglas de negocio aplican y qué criterios permiten considerar la historia como terminada. La especificación se centra en el qué y en el porqué, evitando entrar todavía en decisiones de implementación.

Plan (planificar)

A partir de la especificación, se traduce el qué en un cómo técnico: qué componentes de front-end, back-end e interfaz de usuario se ven afectados, qué modelo de datos se necesita, qué endpoints o contratos hay que definir y cómo encaja esa historia con el resto del sistema ya construido.

Tasks (desglosar en tareas)

El plan técnico se descompone en tareas concretas, pequeñas y verificables, que puedan asignarse, ejecutarse y darse por finalizadas de forma independiente. Cada tarea debe poder trazarse hasta la historia de usuario e incluso hasta el criterio de aceptación que la origina.

Implement (implementar)

Se ejecutan las tareas y se construye el código correspondiente, verificando en todo momento que el resultado obtenido cumple los criterios de aceptación definidos en la fase de especificación.

Loop engineering: iterar hasta quedar satisfechos

Rara vez el resultado de una primera pasada por las cuatro fases es plenamente satisfactorio, y no se espera que lo sea. La práctica exige aplicar loop engineering: cuando la implementación no cumple lo esperado, o cuando al probarla se descubre que la especificación era incompleta o ambigua, el grupo debe volver a la fase que corresponda —revisar la especificación, ajustar el plan, replantear las tareas— y repetir el ciclo, tantas veces como sea necesario, hasta quedar satisfecho con el resultado que se va a entregar.

Se valorará positivamente que esta naturaleza iterativa quede reflejada de forma explícita en el trabajo entregado: versiones sucesivas de una especificación, decisiones técnicas que cambian de rumbo y por qué, tareas que se dividen de nuevo al comprobar que eran demasiado grandes, etc. Iterar no es un síntoma de que algo ha ido mal, sino la forma en que este método de trabajo está pensado para funcionar.

4. Organización del equipo y responsabilidad individual

Aunque el resultado final es una entrega de grupo, la evaluación individual de cada alumno se apoya en un reparto explícito de responsabilidades: cada uno de los cuatro integrantes del grupo se responsabiliza de dos historias de usuario concretas, de entre las identificadas para el sistema asignado.

Responsabilizarse de una historia implica recorrer con ella todo el ciclo: participar en su especificación, contribuir al plan técnico que le afecta, definir o acordar las tareas necesarias y llevar el peso principal de su implementación, sin perjuicio de que el resto del grupo colabore, revise o apoye ese trabajo cuando sea necesario. Cada alumno debe poder explicar, de forma individual, las decisiones tomadas en sus dos historias y defenderlas ante quien evalúe la práctica.

El reparto de las 8 historias entre los 2 miembros del grupo (4 por persona) debe quedar documentado desde el inicio y debe ser trazable en el trabajo entregado: a través de los commits del repositorio, de la autoría de los documentos de especificación y plan, y de las tareas cerradas por cada persona.

5. Fases de la práctica

Fase	Contenido
1. Análisis y especificación	Lectura de la descripción del sistema, identificación de las 8 historias de usuario, redacción de sus especificaciones y reparto de 2 historias por integrante.
2. Planificación técnica	Elaboración del plan técnico de front-end, back-end e interfaz de usuario para cada historia, alineado en la medida de lo posible con la arquitectura propuesta para el sistema.
3. Desglose e implementación	Descomposición en tareas y desarrollo iterativo de la aplicación, aplicando loop engineering cada vez que sea necesario revisar spec, plan o tareas.
4. Refinamiento y verificación	Aplicación de las actividades de clarify, analyze y converge sobre la versión funcional, seguida de la implementación de las correcciones y de su verificación.
5. Cierre y entrega	Consolidación de la aplicación refinada, redacción de la memoria final y preparación de la demostración con evidencias del antes y el después.

5.1 Actividad extra de aclaración con clarify

Una vez disponible la primera versión funcional, el grupo realizará las tres actividades siguientes sobre las ocho historias ya identificadas. Cada alumno revisará sus cuatro historias y otro integrante contrastará sus decisiones. El propósito es mejorar la precisión, la coherencia y el cumplimiento de lo especificado, sin exigir nuevas funcionalidades ajenas al sistema asignado.

Preparación común. Guardar un commit de referencia de la aplicación y sus documentos. Registrar la versión de la herramienta utilizada y seleccionar la funcionalidad activa antes de cada comando. Si varias historias comparten una especificación, ejecutar el comando sobre ella e identificar las historias revisadas. Los ejemplos usan la notación /speckit.clarify, /speckit.analyze y /speckit.converge; se utilizará el nombre equivalente que ofrezca la integración instalada.

Objetivo. Resolver decisiones funcionales que la primera implementación haya dejado implícitas. clarify formula preguntas sobre la especificación e incorpora las respuestas; tras aclararla, se revisarán el plan y las tareas afectadas antes de modificar el código.

1. Probar las cuatro historias asignadas y revisar tres situaciones por historia: el recorrido habitual, una entrada inválida o un caso límite y una condición de permisos o de estado. Si una categoría no aplica, justificarlo y sustituirla por otra relevante.

2. Ejecutar clarify sobre la especificación correspondiente. Responder a sus preguntas a partir de la descripción del sistema y justificar las decisiones del grupo cuando esta no determine una única respuesta. No aceptar una recomendación solo porque la aplicación ya funcione de esa manera.

Ejemplo de instrucción. /speckit.clarify Revisa las ambigüedades de las historias HU-01 y HU-02, especialmente quién puede realizar cada acción, qué datos son válidos y qué ocurre ante estados incompatibles. Formula las preguntas necesarias para concretar criterios de aceptación verificables.

3. Revisar las modificaciones de spec.md y expresar los criterios afectados mediante «Dado… Cuando… Entonces…». Registrar la pregunta, la respuesta, su justificación y la historia afectada. Conservar el informe si no se detectan ambigüedades; no inventar problemas para completar la actividad.

Ejemplo de refinamiento. «El usuario puede modificar un registro» puede concretarse, si el dominio lo requiere, como «Dado un registro cerrado, cuando su propietario intenta modificarlo, el sistema rechaza la operación, explica el motivo y conserva los datos». Es una decisión que debe validarse para el sistema asignado.

Evidencias y finalización. Entregar el registro de decisiones y la diferencia entre la especificación anterior y la revisada. La actividad termina cuando los casos revisados tienen un resultado esperado inequívoco y las dudas que bloqueen la implementación están resueltas. Cada alumno explica al menos una decisión o por qué mantuvo un criterio existente.

5.2 Actividad extra de coherencia con analyze

Objetivo. Comprobar que las especificaciones, los planes y las tareas describen la misma solución. analyze realiza una revisión de coherencia y cobertura entre esos documentos y produce un informe de solo lectura; no corrige por sí mismo las discrepancias ni demuestra que la aplicación funcione.

1. Actualizar los planes y el desglose de tareas que hayan quedado desfasados tras clarify. Mantener los identificadores de las historias y de los criterios para poder seguir cada cambio. Revisar también las dependencias entre historias de distintos integrantes.

2. Ejecutar analyze con spec.md, plan.md y tasks.md disponibles para la funcionalidad activa. Conservar el primer informe antes de corregir sus hallazgos.

Ejemplo de instrucción. /speckit.analyze Revisa la coherencia entre spec.md, plan.md y tasks.md. Presta atención a criterios sin tareas, tareas sin requisito que las justifique, reglas contradictorias y dependencias entre historias. Informa de la gravedad y de los documentos afectados.

3. Contrastar cada hallazgo con el material del proyecto. Elaborar un registro con identificador, historia o criterio, gravedad, evidencia, decisión, responsable y estado. Justificar los falsos positivos y las cuestiones no aplicables; no copiar el informe sin revisión.

4. Corregir explícitamente la especificación, el plan o las tareas que corresponda. Una omisión del plan no se resuelve eliminando un requisito válido. Si el hallazgo revela una decisión funcional pendiente, volver a clarify antes de ajustar la solución técnica.

5. Construir una matriz de trazabilidad para las ocho historias. Cada fila debe relacionar una historia y criterio con la decisión del plan, las tareas que lo cubren y la prueba prevista. Otro integrante revisará las filas de cada alumno. Repetir analyze después de los ajustes.

Ejemplo de hallazgo. Una historia exige impedir la edición de registros cerrados, pero el plan solo prevé desactivar un botón y no existe una tarea de validación en el servidor. El grupo debe completar el plan y las tareas, incluyendo una comprobación de rechazo de la operación y de conservación de los datos.

Evidencias y finalización. Entregar los informes inicial y final, el registro de hallazgos y la matriz de trazabilidad. No deben quedar contradicciones críticas o de gravedad alta sin resolver, ni criterios del alcance comprometido sin tareas y prueba prevista. Una excepción requiere una justificación explícita y, si cambia el alcance exigido, validación del docente.

Revisión entre compañeros. El revisor seleccionará al menos un criterio de cada historia de su compañero y comprobará que la tarea propuesta permitiría verificarlo. Si no se detectan incidencias reales, se documentará la cobertura comprobada; no se introducirán errores artificiales.

5.3 Actividad extra de cierre de brechas con converge

Objetivo. Comprobar qué obligaciones de spec.md, plan.md y tasks.md siguen sin cumplirse en el código. converge se ejecuta después de implement y añade al final de tasks.md las tareas de corrección necesarias. No modifica el código, la especificación ni el plan; las nuevas tareas se ejecutan mediante implement.

1. Implementar las tareas pendientes derivadas de la revisión anterior y ejecutar las pruebas disponibles. Guardar el estado de partida y seleccionar la funcionalidad que se va a evaluar.

2. Ejecutar converge y revisar las evidencias de cada hallazgo. Diferenciar trabajo ausente, cumplimiento parcial, comportamiento contradictorio y código sin justificación en los documentos. No considerar una tarea marcada como completada como prueba suficiente de cumplimiento.

Ejemplo de instrucción. /speckit.converge Contrasta la implementación actual con spec.md, plan.md y tasks.md. Revisa los criterios de aceptación, los casos de error y las obligaciones técnicas documentadas. Identifica el trabajo pendiente y añade tareas trazables para completarlo.

3. Relacionar cada nueva tarea con la historia, el criterio o la obligación técnica de origen y asignarle responsable y revisor. Si la discrepancia procede de un requisito ambiguo, volver a clarify y analyze antes de implementar. No ampliar el alcance ni rebajar un criterio únicamente para obtener un informe sin incidencias.

4. Ejecutar implement sobre las tareas correctoras y comprobar su resultado. Para cada corrección, conservar una prueba reproducible del fallo previo y del resultado posterior, con datos de entrada y resultado esperado y observado. Si no puede ejecutarse la versión anterior, indicar qué evidencia acredita el defecto.

5. Repetir converge tras las correcciones. Si aparecen nuevas brechas válidas, continuar el ciclo converge → implement → pruebas → converge. Conservar los informes y el historial de tareas sin borrar las iteraciones anteriores.

Ejemplo de corrección. La interfaz impide editar un registro cerrado, pero una petición directa aún lo modifica. Si la especificación prohíbe esa modificación, se implementará la validación necesaria y se verificará tanto el rechazo como la ausencia de cambios en los datos.

Evidencias y finalización. Entregar los informes de convergencia, las tareas añadidas, los commits de corrección y los resultados de las pruebas. El cierre exige que no queden brechas accionables del alcance comprometido y que los criterios de aceptación se hayan verificado. Un resultado limpio de converge no sustituye las pruebas ni la demostración funcional.

Si no hay brechas. Conservar el informe y demostrar, con las mismas comprobaciones de aceptación, que la implementación ya satisface el alcance revisado. Se evalúa la calidad de la verificación y de las decisiones, no la cantidad de defectos encontrados.

5.4 Verificación integrada y valoración del refinamiento

El grupo cerrará el refinamiento mediante una revisión conjunta de la aplicación. Cada integrante mantiene la responsabilidad sobre sus dos historias y debe poder reconstruir el recorrido desde una decisión funcional hasta su comprobación en la versión final.

Recorridos completos. Ejecutar al menos dos recorridos que conecten historias de distintos integrantes. Incluir un recorrido satisfactorio y otro con una condición de error, permiso o estado. Registrar precondiciones, datos de prueba, pasos, resultado esperado, resultado observado y versión del código.

Regresión. Volver a comprobar los criterios de las historias afectadas por las correcciones y de aquellas que dependan de ellas. Cuando exista una suite de pruebas, ejecutarla y conservar su resultado; completar con verificaciones manuales los aspectos que no cubra.

Balance de la iteración. Comparar la versión inicial y la final: ambigüedades resueltas, incidencias documentales abiertas y cerradas, brechas de implementación pendientes y criterios verificados sobre el total revisado. Indicar los denominadores y enlazar las evidencias. Estos indicadores describen la mejora; no se premiará encontrar más defectos.

Registro mínimo. La memoria incluirá una tabla con los campos: historia y criterio, hallazgo o decisión, comando utilizado, documentos o código afectados, tarea y commit, prueba y resultado, responsable, revisor y estado. Las salidas completas de los comandos pueden conservarse en el repositorio y enlazarse desde la memoria.

6. Entregables

Se entregarán 2 ficheros zip (max tamaño de subida 100 Mb entre los dos!!!):

1. ProyectoSSD_GXX.zip: Es la carpeta del proyecto realizado, debe contener una estructura parecida a `biblioteca.zip`
    a. Fuentes del código generado.
    b. Carpeta **docs** con dos ficheros markdown `agrupamiento.md` y `memoria.md` el primero presenta el reparto de historias por alumno y el segundo la memoria del proyecto, es obligatorio que siga el índice propuesto en el sistema ejemplo biblioteca (ver índice de la `memoria.md`).
        i. Ejemplo de prompt para memoria.md: `genera la memoria del proyecto en docs/memoria.md siguiendo esta estructura: Portada y resumen: qué es, para quién y el resultado en un párrafo. URL repositorio GITHUB: referencia a la URL del repo. Introducción: contexto, problema que resuelve y objetivos. Análisis: historias de usuario, requisitos funcionales y no funcionales, actores y roles. Con Spec Kit sale casi directamente de los spec.md. Diseño: arquitectura (en un monorepo, apps/ y packages/), modelo de datos, diagramas, decisiones técnicas y por qué se tomaron. Sale de los plan.md, data-model.md y de la constitución. Implementación: pila tecnológica, estructura del repositorio, partes más relevantes del código y decisiones que se tomaron sobre la marcha. Pruebas y validación: estrategia de tests (unitarios, e2e), resultados y validación manual. Despliegue y uso: cómo se instala y se ejecuta (docker-compose, .env.example) y un manual de usuario breve. Planificación y seguimiento: iteraciones, issues, PR e historial de commits. Conclusiones: qué se consiguió, problemas encontrados y trabajo futuro. Anexos: enlace al repositorio, specs completas y glosario.`

c. Carpeta **specs** con el resultado de las especificaciones desarrolladas siguiendo GITHUB spec kit. Cada especificación ha de tener todos los ficheros markdown resultado de aplicar el ciclo specify-clarify-plan-tasks-analyze-implement-converge. La `constitution.md` estará en su carpeta correspondiente (.specify).

2. DemoSSD_GXX.zip: Carpeta con la demo del proyecto en formato `.mp4`, debe contener una estructura parecida a `demo_biblioteca.zip`.

a. Ejemplo de prompt: `quiero que me hagas un video mostrando todas las historias de usuario y todos los posibles escenarios. La demo no debe de ser rápida sino lo suficientemente dinámica para que se pueda apreciar bien la funcionalidad del sistema. Móntame también una página desde donde pueda acceder al video y a la reproducción de las historias.`

7. Normas generales

- El uso de herramientas de inteligencia artificial y de asistentes de codificación agéntica está permitido y forma parte del enfoque de spec-driven development propuesto en esta práctica; lo que se evalúa no es si se ha usado IA, sino la calidad de las especificaciones, las decisiones tomadas y la capacidad de cada alumno de explicar y defender el trabajo del que se ha responsabilizado.
- Cada alumno debe poder justificar individualmente, en cualquier momento de la evaluación, las decisiones tomadas en las historias de usuario que le corresponden.
- El reparto de historias entre los integrantes del grupo debe entregarse desde el inicio de la práctica y debe mantenerse trazable a lo largo de todo el trabajo.